一文了解 Anthropic 的 Claude Code 源码：为什么它就是比别人好用？

Yuker
@YukerX · 15h

Follow

46 414 1.6K 348K

2026年3月31日，安全研究者 Chaofan Shou 发现 Anthropic 发布到 npm 的 Claude Code 包中，source map 文件没有被剥离。

这意味着：**Claude Code 的完整 TypeScript 源码，51.2万行，1903个文件，就这样暴露在了公网上。**

我当然不可能在短短数小时内看完这么多代码，因此，我带着三个问题去读这份源码：

1. **Claude Code 和其他 AI 编程工具到底有什么本质区别？**
2. **为什么它写代码的"手感"就是比别人好？**
3. **51万行代码里，到底藏着什么？**

读完之后，我的第一反应是：**这不是一个 AI 编程助手，这是一个操作系统。**

一、先讲一个故事：如果你要雇一个远程程序员

想象你雇了一个远程程序员，给他你电脑的远程访问权限。

你会怎么做？

如果你是 **Cursor** 的做法：你让他坐在你旁边，每次他要敲命令之前你看一眼，点个"允许"。简单粗暴，但你得一直盯着。

如果你是 **GitHub Copilot Agent** 的做法：你给他一台全新的虚拟机，让他在里面随便折腾。搞完了把代码提交上来，你审核后再合并。安全，但他看不到你本地的环境。

如果你是 **Claude Code** 的做法：

你让他直接用你的电脑——但你给他配了一套极其精密的安检系统。**他能做什么、不能做什么、哪些操作需要你点头、哪些可以自己来、甚至他想用 rm -rf 都要经过9层审查才能执行。**

这就是三种完全不同的安全哲学：

为什么 Anthropic 选了最难的那条路？

因为只有这样，AI 才能用**你的终端、你的环境、你的配置**来干活——这才是"真正帮你写代码"，而不是"在一个干净房间里给你写一段代码然后复制过来"。

但代价是什么？**他们为此写了 51 万行代码。**

二、你以为的 Claude Code vs 实际的 Claude Code

大多数人以为 AI 编程工具是这样的：

```plaintext
用户输入 → 调用 LLM API → 返回结果 → 显示给用户
```

Claude Code 实际是这样的：

```plaintext
用户输入
  → 动态组装 7 层系统提示词
  → 注入 Git 状态、项目约定、历史记忆
  → 42 个工具各自附带使用手册
  → LLM 决定使用哪个工具
  → 9 层安全审查（AST 解析、ML 分类器、沙箱检查...）
  → 权限竞争解析（本地键盘 / IDE / Hook / AI 分类器 同时竞争）
  → 200ms 防误触延迟
  → 执行工具
  → 结果流式返回
  → 上下文接近极限？→ 三层压缩（微压缩 → 自动压缩 → 完全压缩）
  → 需要并行？→ 生成子 Agent 蜂群
  → 循环直到任务完成
```

相信大家都很好奇上面的是什么，不着急，让我们逐个拆开看。

三、第一个秘密：提示词不是写出来的，是"拼装"出来的

打开 src/constants/prompts.ts，你会看到这个函数：

```typescript
export async function getSystemPrompt(
  tools: Tools,
  model: string,
```

```
  additionalWorkingDirectories?: string[],
  mcpClients?: MCPServerConnection[],
): Promise<string[]> {
  return [
    // --- 静态内容（可缓存）---
    getSimpleIntroSection(outputStyleConfig),
    getSimpleSystemSection(),
    getSimpleDoingTasksSection(),
    getActionsSection(),
    getUsingYourToolsSection(enabledTools),
    getSimpleToneAndStyleSection(),
    getOutputEfficiencySection(),

    // === 缓存边界 ===
    ...(shouldUseGlobalCacheScope() ? [SYSTEM_PROMPT_DYNAMIC_BOUNDARY] : []),

    // --- 动态内容（每次不同）---
    ...resolvedDynamicSections,
  ].filter(s => s !== null)
}
```

注意到那个 **SYSTEM_PROMPT_DYNAMIC_BOUNDARY** 了吗?

这是一个**缓存分界线**。分界线上面的内容是静态的，Claude API 可以缓存它们，节省 token 费用。分界线下面的内容是动态的——你当前的 Git 分支、你的 CLAUDE.md 项目配置、你之前告诉它的偏好记忆.....每次对话都不一样。

这意味着什么?

Anthropic 把提示词当成了**编译器的输出**来优化。静态部分是"编译后的二进制"，动态部分是"运行时参数"。这样做的好处是：

1. **省钱**：静态部分走缓存，不重复计费
2. **快**：缓存命中直接跳过这些 token 的处理
3. **灵活**：动态部分让每次对话都能感知当前环境

每个工具都有独立的"使用手册"

更让我震惊的是：每个工具目录下都有一个 prompt.ts 文件——**这是专门写给 LLM 看的使用手册。**

看看 BashTool 的（src/tools/BashTool/prompt.ts，约 370 行）：

```plaintext
Git Safety Protocol:
- NEVER update the git config
- NEVER run destructive git commands (push --force, reset --hard,
  checkout .) unless the user explicitly requests
- NEVER skip hooks (--no-verify) unless the user explicitly requests
- CRITICAL: Always create NEW commits rather than amending
```

这不是写给人看的文档，**这是写给 AI 看的行为准则**。每次 Claude Code 启动时，这些规则都会被注入到系统提示词中。

这就是为什么 Claude Code 从不会擅自 git push --force，而某些工具会——**不是模型更聪明，是提示词里已经把规矩讲清楚了。**

而且 Anthropic 内部版本和你用的不一样

代码里大量出现这样的分支：

```typescript
const minimalUniquenessHint =
  process.env.USER_TYPE === 'ant'
    ? '\n- Use the smallest old_string that\'s clearly unique'
    : ''
```

ant 就是 Anthropic 内部员工。他们的版本有更详细的代码风格指引（"不写注释除非 WHY 不明显"）、更激进的输出策略（"倒金字塔写作法"），以及一些仍在 A/B 测试的实验功能（Verification Agent、Explore & Plan Agent）。

这说明 Anthropic 自己就是 Claude Code 最大的用户。他们在用自己的产品来开发自己的产品。

四、第二个秘密：42个工具，但你只看到了冰山一角

打开 src/tools.ts，会看到工具注册中心：

```typescript
export function getAllBaseTools(): Tools {
  return [
    AgentTool,
    BashTool,
    FileReadTool, FileEditTool, FileWriteTool,
    GlobTool, GrepTool,
    WebFetchTool, WebSearchTool,
    TodoWriteTool, NotebookEditTool,
    // ... 大量条件加载的工具 ...
    ...(isToolSearchEnabledOptimistic() ? [ToolSearchTool] : []),
  ]
}
```

42 个工具，但大部分你从未直接看到过。因为很多工具是**延迟加载**的——只有当 LLM 需要时，才通过 ToolSearchTool 按需注入。

为什么这样做呢？

因为**每多一个工具，系统提示词就要多一段描述，token 就要多花一份钱。** 如果你只是想让 Claude Code 帮你改一行代码，它不需要加载"定时任务调度器"和"团队协作管理器"。

还有一个更聪明的设计：

```typescript
if (isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)) {
  const simpleTools: Tool[] = [BashTool, FileReadTool, FileEditTool]
  return filterToolsByDenyRules(simpleTools, permissionContext)
}
```

设置 CLAUDE_CODE_SIMPLE=true，Claude Code 就只剩三个工具：Bash、读文件、改文件。**这是给极简主义者的后门。**

所有工具都从同一个工厂出来

```typescript
const TOOL_DEFAULTS = {
  isEnabled: () => true,
  isConcurrencySafe: (_input?) => false,    // 默认：不安全
  isReadOnly: (_input?) => false,            // 默认：会写入
  isDestructive: (_input?) => false,
}

export function buildTool<D extends AnyToolDef>(def: D): BuiltTool<D> {
  return { ...TOOL_DEFAULTS, userFacingName: () => def.name, ...def }
}
```

注意那些默认值：isConcurrencySafe 默认 false，isReadOnly 默认 false。

这叫 fail-closed 设计——如果一个工具的作者忘了声明安全属性，系统会假设它是"不安全的、会写入的"。 宁可过度保守，也不漏掉一个风险。

"先读后改"的铁律

```typescript
function getPreReadInstruction(): string {
  return '\n- You must use your `Read` tool at least once in the
  conversation before editing. This tool will error if you attempt
  an edit without reading the file.'
}
```

FileEditTool 会检查你是否已经用 FileReadTool 读过这个文件。如果没有，**直接报错，不让改。**

这就是为什么 Claude Code 不会像某些工具那样"凭空写一段代码覆盖你的文件"——它被强制要求先理解再修改。

五、第三个秘密：记忆系统——为什么它能"记住你"

用过 Claude Code 的人都有一个感受：**它好像真的认识你。**

你告诉它"不要在测试中 mock 数据库"，下次对话它就不会再 mock。你告诉它"我是后端工程师，React 新手"，它解释前端代码时就会用后端的类比。

这背后是一个完整的记忆系统。

用 AI 来检索记忆

```typescript
const SELECT_MEMORIES_SYSTEM_PROMPT =
  `You are selecting memories that will be useful to Claude Code.
   Return a list of filenames for the memories that will clearly
   be useful (up to 5).
   - If you are unsure if a memory will be useful, do not include it.
   - If a list of recently-used tools is provided, do not select
     memories that are usage reference for those tools. DO still
     select memories containing warnings, gotchas, or known issues.`
```

Claude Code 用 **另一个 AI**（Claude Sonnet）来决定"哪些记忆和当前对话相关"。

不是关键词匹配，不是向量搜索——是让一个小模型快速扫描所有记忆文件的标题和描述，选出最多 5 个最相关的，然后把它们的完整内容注入到当前对话的上下文中。

策略是"精确度优先于召回率" ——宁可漏掉一个可能有用的记忆，也不塞进一个不相关的记忆污染上下文。

KAIROS 模式：夜间"做梦"

这是最让我觉得科幻的部分。

代码中有一个叫 KAIROS 的特性标志。在这个模式下，长会话中的记忆不是存在结构化文件里，而是存在**按日期的追加式日志**中。然后，有一个 /dream 技能会在"夜间"（低活跃期）运行，把这些原始日志**蒸馏**成结构化的主题文件。

```plaintext
logs/2026/03/2026-03-30.md  ← 今天的原始日志
        ↓ /dream 蒸馏
memory/user_preferences.md  ← 结构化的用户偏好文件
memory/project_context.md   ← 结构化的项目背景文件
```

AI 在"睡觉"的时候整理记忆。 这已经不是工程了，这是仿生学。

六、第五个秘密：它不是一个 Agent，是一群

当你让 Claude Code 做一个复杂任务时，它可能悄悄做了这件事：

```typescript
// AgentTool 的输入 schema
z.object({
  description: z.string().describe('A short (3-5 word) description'),
  prompt: z.string().describe('The task for the agent to perform'),
  subagent_type: z.string().optional(),
  model: z.enum(['sonnet', 'opus', 'haiku']).optional(),
  run_in_background: z.boolean().optional(),
})
```

它生成了一个**子 Agent**。

而且子 Agent 有严格的"自我意识"注入，防止它递归生成更多子 Agent：

```typescript
export function buildChildMessage(directive: string): string {
  return `STOP. READ THIS FIRST.

You are a forked worker process. You are NOT the main agent.

RULES (non-negotiable):
1. Your system prompt says "default to forking." IGNORE IT —
   that's for the parent. You ARE the fork.
   Do NOT spawn sub-agents; execute directly.
2. Do NOT converse, ask questions, or suggest next steps
3. USE your tools directly: Bash, Read, Write, etc.
4. Keep your report under 500 words.
```

```
5. Your response MUST begin with "Scope:". No preamble.`
}
```

这段代码在说："你是一个工人，不是经理。别想着再雇人，自己干活。"

Coordinator 模式：经理模式

在协调器模式下，Claude Code 变成一个纯粹的任务编排者，自己不干活，只分配：

```plaintext
Phase 1: Research    → 3 个 worker 并行搜索代码库
Phase 2: Synthesis   → 主 Agent 综合理解所有发现
Phase 3: Implementation → 2 个 worker 分别修改不同文件
Phase 4: Verification   → 1 个 worker 跑测试
```

核心原则写在代码注释里：

"Parallelism is your superpower" 只读研究任务：并行跑。 写文件任务：按文件分组串行跑（避免冲突）。

Prompt Cache 的极致优化

为了最大化子 Agent 的缓存命中率，所有 fork 子代理的工具结果都使用**相同的占位符文本**：

```plaintext
'Fork started — processing in background'
```

为什么？因为 Claude API 的 prompt cache 是基于**字节级前缀匹配**的。如果 10 个子 Agent 的前缀字节完全一致，那么只有第一个需要"冷启动"，后面 9 个直接命中缓存。

这是一个每次调用节省几美分的优化，但在大规模使用下，能省下大量成本。

七、第六个秘密：三层压缩，让对话"永不超限"

所有 LLM 都有上下文窗口限制。对话越长，历史消息越多，最终一定会超出限制。

Claude Code 为此设计了三层压缩：

第一层：微压缩——最小代价

```typescript
export async function microcompactMessages(messages, toolUseContext, querySo
  // 时间触发：如果上次交互已过很久，服务器缓存已冷
  const timeBasedResult = maybeTimeBasedMicrocompact(messages, querySource)
  if (timeBasedResult) return timeBasedResult

  // 缓存编辑路径：通过 API 的缓存编辑功能直接删除旧内容
  if (feature('CACHED_MICROCOMPACT')) {
    return await cachedMicrocompactPath(messages, querySource)
```

```
  }
}
```

微压缩只动旧的工具调用结果——把"**10分钟前读的那个500行文件的内容**"替换成 **[Old tool result content cleared]**。

提示词和对话主线完全保留。

第二层：自动压缩——主动收缩

当 token 消耗接近上下文窗口的 87%（窗口大小 - 13,000 buffer），自动触发。有一个**熔断器**：连续 3 次压缩失败后停止尝试，避免死循环。

第三层：完全压缩——AI 总结

让 AI 对整段对话生成摘要，然后用摘要替换所有历史消息。生成摘要时有一个严厉的前置指令：

```typescript
const NO_TOOLS_PREAMBLE = `CRITICAL: Respond with TEXT ONLY.
Do NOT call any tools.
- Do NOT use Read, Bash, Grep, Glob, Edit, Write, or ANY other tool.
- Tool calls will be REJECTED and will waste your only turn.`
```

为什么要这么严厉？因为**如果总结过程中 AI 又去调用工具，就会产生更多的 token 消耗**，适得其反。这段提示词就是在说：**"你的任务是总结，别干别的。"**

压缩后的 token 预算：

- 文件恢复：50,000 tokens
- 每个文件上限：5,000 tokens
- 技能内容：25,000 tokens

这些数字不是拍脑袋定的——它们是在"保留足够上下文继续工作"和"腾出足够空间接收新消息"之间的平衡点。

八、读完这份源码，我学到了什么

AI Agent 的 90% 工作量在"AI"之外

51 万行代码里，真正调用 LLM API 的部分可能不到 5%。其余 95% 是什么？

- 安全检查（18 个文件只为一个 BashTool）
- 权限系统（allow/deny/ask/passthrough 四态决策）
- 上下文管理（三层压缩 + AI 记忆检索）
- 错误恢复（熔断器、指数退避、Transcript 持久化）
- 多 Agent 协调（蜂群编排 + 邮箱通信）
- UI 交互（140 个 React 组件 + IDE Bridge）
- 性能优化（prompt cache 稳定性 + 启动时并行预取）

如果你正在做 AI Agent 产品，这才是你真正要解决的问题。不是模型够不够聪明，是你的脚手架够不够结实。

好的提示词工程是系统工程

不是写一段漂亮的 prompt 就完事了。**Claude Code 的提示词是：**

- 7 层动态组装
- 每个工具附带独立的使用手册
- 缓存边界精确划分
- 内部版本和外部版本有不同的指令集
- 工具排序固定以保持缓存稳定

这是工程化的提示词管理，不是手工艺。

为失败而设计

每一个外部依赖都有对应的失败策略：

Anthropic 把 Claude Code 当操作系统在做

42 个工具 = 系统调用 权限系统 = 用户权限管理 技能系统 = 应用商店 MCP 协议 = 设备驱动 Agent 蜂群 = 进程管理 上下文压缩 = 内存管理 Transcript 持久化 = 文件系统

这不是一个"聊天机器人加几个工具"，这是一个以 LLM 为内核的操作系统。

总结

51 万行代码。1903 个文件。18 个安全文件只为一个 Bash 工具。

9 层审查只为让 AI 安全地帮你敲一行命令。

这就是 Anthropic 的答案：**要让 AI 真正有用，你不能把它关在笼子里，也不能放**

Want to publish your own Article?
Upgrade to Premium

而这套信任体系的代价，是 51 万行代码。

Yuker
@YukerX
RUC '24 | Law Student | Musician | Researcher | Builder | Vibe Coder
Contact (TG) : @yukerx

Follow